

Curso de Engenharia Informática
Cadeira de Estrutura de Dados II — Semestre 2025/2

Algoritmos de ordenação, busca e estruturas de dados aplicados à gestão acadêmica em linguagem C

Autores (preencher):

Nome do estudante	Nº de matrícula
_____	_____
_____	_____
_____	_____
_____	_____

1. Introdução

O presente relatório descreve a concepção e a implementação de um **Sistema de Gestão de Notas**, desenvolvido como trabalho final da cadeira de Estrutura de Dados II. O problema proposto consiste em modelar, de forma computacional, o ambiente académico de um professor que gere vários anos lectivos, cada um com as suas turmas, grupos de trabalho, alunos e respectivas notas, oferecendo ainda mecanismos de pesquisa, ordenação, comunicação e persistência de dados.

A aplicação foi desenvolvida integralmente em linguagem **C**, recorrendo ao conceito de **Tipos Abstractos de Dados (TAD)**: cada estrutura de dados foi encapsulada num par de ficheiros de cabeçalho e de implementação, separando claramente a interface da lógica interna. O objectivo central foi materializar, numa solução prática e coesa, os conceitos de algoritmos de ordenação, algoritmos de busca e estruturas de dados estudados ao longo do semestre.

O sistema permite registar e remover entidades a todos os níveis da hierarquia, calcular médias, determinar a aprovação ou reprovação, produzir relatórios por turma, ordenar alunos por mérito ou por nome, trocar mensagens entre o professor e os grupos e, por fim, gravar e recuperar todo o estado em ficheiro.

2. Metodologia

2.1. Organização e divisão do trabalho

O trabalho foi decomposto por módulos, fazendo coincidir cada estrutura de dados com um par de ficheiros `.h` / `.c`. Esta divisão permitiu o desenvolvimento em paralelo, a testagem isolada de cada componente e uma integração final simplificada através do módulo principal. A atribuição de responsabilidades encontra-se no quadro da secção 2.6.

2.2. Estruturas de dados utilizadas e justificação

A escolha de cada estrutura foi orientada pela operação dominante em cada nível da hierarquia (busca, percurso ordenado ou inserção sequencial):

Entidade	Estrutura	Justificação
Anos lectivos	Árvore AVL	Busca por ano em tempo logarítmico; balanceamento garante a eficiência.
Turmas	Árvore AVL	Mesma necessidade de busca eficiente por identificador, dentro de cada ano.
Grupos	Lista duplamente ligada	Número reduzido e percurso sequencial; remoção em ambos os sentidos.
Alunos	Lista duplamente ligada	Pertencem a um grupo; navegação anterior/seguite facilita a remoção.
Notas	Árvore BST	Busca e percurso ordenado das notas por nome de disciplina.
Índice de alunos	Árvore ternária (TST)	Pesquisa de alunos por nome (cadeia de caracteres) de forma eficiente.
Mensagens	Fila (FIFO)	As mensagens são lidas pela ordem de chegada, característica de uma fila.
Ranking	Quicksort / Merge sort	Ordenação de um vector de alunos por média (Quicksort) ou nome (Merge sort).

O modelo de propriedade (*ownership*) é hierárquico: o professor possui os anos, cada ano possui as turmas, cada turma possui os grupos, cada grupo possui os alunos e cada aluno possui as suas notas. A

árvore ternária guarda apenas **ponteiros** para os alunos, nunca os liberta, evitando dupla libertação de memória.

2.3. Algoritmos de ordenação

Para a funcionalidade de ranking, os alunos de todos os grupos de uma turma são recolhidos para um vector dinâmico, calculando-se a média uma única vez por aluno. Sobre esse vector aplicam-se dois algoritmos clássicos:

- **Quicksort** — utilizado na ordenação por média (ranking de mérito). Baseia-se no particionamento em torno de um pivot, com complexidade média $O(n \log n)$.
- **Merge sort** — utilizado na ordenação alfabética por nome. Aplica a estratégia de divisão e conquista, é estável e garante $O(n \log n)$ no pior caso.

A utilização propositada de dois algoritmos distintos serve para demonstrar duas estratégias de ordenação estudadas na cadeira, comparando particionamento (Quicksort) com intercalação (Merge sort).

2.4. Bibliotecas e ficheiros de dados

Foram utilizadas apenas bibliotecas padrão da linguagem C (`stdio.h`, `stdlib.h`, `string.h`), não havendo dependências externas, o que torna o projecto portátil e de compilação imediata. A persistência é assegurada por ficheiros de texto: um formato simples, com uma entidade por linha e o carácter ‘;’ como separador, permite gravar (opção 12) e carregar (opção 13) todo o estado do sistema. O programa lê ainda um ficheiro de entrada de teste (`entrada_teste.txt`) e escreve relatórios e rankings em ficheiros de saída.

2.5. Decisões de implementação e referências consultadas

- Validação de notas no intervalo de 0 a 20, com mensagem de falha explícita.
- Mensagens de sucesso e de erro em todas as operações do menu.
- Gestão rigorosa de memória: toda a memória alocada é libertada em cascata ao encerrar; verificado com a ferramenta *valgrind* (sem fugas nem acessos inválidos).
- Compilação sem avisos com as opções `-Wall -Wextra`.
- Como referência de implementação das árvores AVL e dos algoritmos de ordenação foram consultadas as obras de Cormen et al. e de Ziviani, indicadas na bibliografia.

2.6. Responsabilidades de cada membro (preencher)

A tabela seguinte propõe uma divisão por módulos; deve ser ajustada com os nomes reais dos membros do grupo, conforme exigido no enunciado.

Membro (nome)	Módulos / responsabilidades	Parte da defesa
_____	estruturas.h, listas (grupos e alunos)	Listas ligadas
_____	AVL (anos e turmas), balanceamento	Árvores AVL

_____	BST (notas) e TST (índice de alunos)	BST e TST
_____	ordenação (ranking), relatórios	Quicksort / Merge sort
_____	ficheiro (persistência) e main (menu)	Persistência e interface

3. Funcionalidades do programa

A interface é um menu de consola com as seguintes operações:

- Registo e remoção de anos lectivos, turmas, grupos, alunos e notas;
- Lançamento de notas individuais ou de grupo (propagadas a todos os membros);
- Cálculo de média e estado de aprovação de um aluno (pesquisa por nome na TST);
- Impressão e gravação de relatórios por turma;
- Ranking de alunos por média (Quicksort) ou por nome (Merge sort), no ecrã ou em ficheiro;
- Envio e leitura de mensagens entre o professor e os grupos;
- Gravação e carregamento de todo o estado em ficheiro.

4. Conclusão

A realização deste trabalho permitiu consolidar, numa única aplicação, os três pilares da cadeira: estruturas de dados, algoritmos de busca e algoritmos de ordenação. A modelação da hierarquia académica evidenciou como a escolha da estrutura certa para cada nível (árvores para busca eficiente, listas para sequências, fila para mensagens) influencia directamente a clareza e o desempenho do código.

As principais dificuldades surgiram na **gestão de memória** — em particular na remoção em árvores AVL com nós de dois filhos, onde foi necessário transferir cuidadosamente a propriedade das sub-estruturas para evitar fugas e duplas libertações — e na coordenação entre o índice ternário e as listas de alunos durante as remoções. O recurso ao *valgrind* foi decisivo para validar a correcção da gestão de memória.

Em síntese, o grupo considera que os objectivos foram alcançados: o sistema é funcional, está modularizado segundo o conceito de Tipos Abstractos de Dados, compila sem avisos e executa sem fugas de memória.

5. Bibliografia

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

ZIVIANI, N. **Projeto de algoritmos: com implementações em Pascal e C**. 3. ed. São Paulo: Cengage Learning, 2011.

TENENBAUM, A. M.; LANGSAM, Y.; AUGENSTEIN, M. J. **Estruturas de dados usando C**. São Paulo: Pearson Makron Books, 1995.

SEGEWICK, R. **Algorithms in C, Parts 1–4: Fundamentals, Data Structures, Sorting, Searching**. 3rd ed. Boston: Addison-Wesley, 1998.

KERNIGHAN, B. W.; RITCHIE, D. M. **A linguagem de programação C: padrão ANSI**. Rio de Janeiro: Campus, 1989.

CPPREFERENCE. **C reference**. Disponível em: <<https://en.cppreference.com/w/c>>. Acesso em: jun. 2026.